Train Booking System

Project Documentation

Object Oriented Software Development CA3

South East Technological University

Department of Computing

Lecturer: Dr. Jason Barron

Student Name: Javier Reyes

Student ID: C00312829

Date: April 2026

Table of Contents

1. Description
2. Requirements
   2.1 Functional Requirements
   2.2 Non-Functional Requirements
   2.3 Technologies Used
3. Database Design
   3.1 ER Diagram
   3.2 Database Tables (Structure & Data)
4. Application Architecture
5. GUI Design
6. Interesting Source Code Snippets
   6.1 Singleton Database Manager
   6.2 Password Hashing with SHA-256
   6.3 Seat Grid with Anonymous Inner Class Listener
   6.4 Inner Join for Booking Details
   6.5 Input Validation and Error Handling
   6.6 Finally Block for Resource Cleanup
7. Tests
   7.1 JUnit Unit Tests
   7.2 Manual Test Cases
9. References

1. Description

This project is a train booking system built in Java. The app lets users register, log in, search for trains, pick a seat and make or cancel bookings. I used Java Swing for the interface and SQLite as the database, connected using JDBC.

The data is stored in a local SQLite database file that gets created automatically when you first run the app. Users can search for trains by route and date, select a seat from a visual seat map, confirm a booking and view or cancel their bookings.

I split the code into separate packages for models, database, services and GUI to keep things organised and easier to work with.

Key features of the application include:

- User registration and login with SHA-256 password hashing
- Train search with dropdown station selection and date filtering
- Visual seat selection grid with colour-coded seat classes (First, Second, Standard)
- Full CRUD operations: Create bookings, Read/retrieve booking history, Update seat availability, Delete/cancel bookings
- Input validation and error handling throughout the application
- Booking filtering by status (All, Confirmed, Cancelled)
- Confirmation dialogs before booking and cancellation actions
- JUnit 5 unit tests for service layer validation and price calculation

2. Requirements

2.1 Functional Requirements

ID	Requirement	Description
FR 1	User Registration	Users can create an account with username, password, full name, and email. Duplicate usernames and emails are rejected.
FR 2	User Login	Registered users can log in with their username and password. Passwords are verified against stored SHA-256 hashes.
FR 3	Train Search	Users can search for trains by selecting origin station, destination station, and travel date from dropdown menus.
FR 4	Seat Selection	Users can view all seats for a chosen train in a visual grid, colour-coded by class and availability, and select a seat.
FR 5	Create Booking	Users can book a selected seat. The system marks the seat as booked, calculates the price based on seat class, and stores the booking.
FR 6	View Bookings	Users can view all their bookings in a table with filtering options for All, Confirmed, and Cancelled bookings.
FR 7	Cancel Booking	Users can cancel a confirmed booking. The seat is freed and the booking status is set to CANCELLED.
FR 8	Input Validation	All user inputs are validated: empty fields, password length, email format, matching passwords, and same origin/destination checks.

2.2 Non-Functional Requirements

- The application must run on any system with Java 17+ and the SQLite JDBC driver.
- The GUI must be responsive and user-friendly, with clear feedback messages for all actions.
- Passwords must never be stored in plain text; SHA-256 hashing is used.
- The database must enforce referential integrity using foreign keys.
- All database operations must use PreparedStatements to prevent SQL injection.
- Database resources (PreparedStatement, ResultSet) should be closed in finally blocks where appropriate.

2.3 Technologies Used

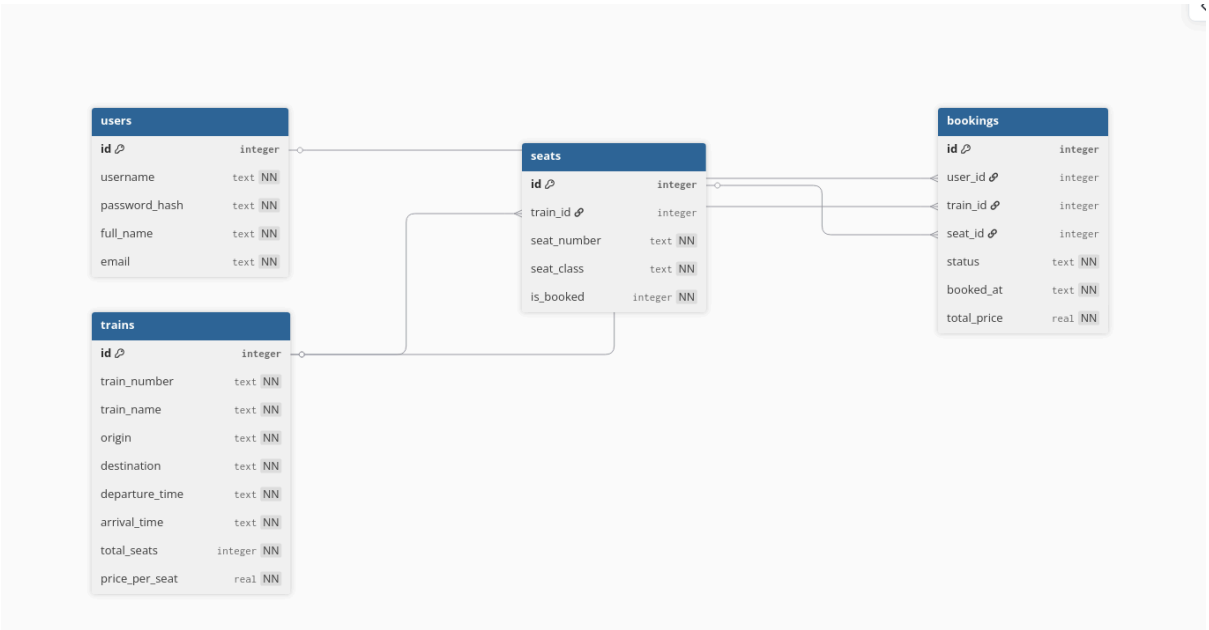
Technology	Purpose
Java 17+	Core programming language
Java Swing	GUI framework (JFrame, JPanel, JTable, JButton, JComboBox, JTabbedPane, JOptionPane, JTextField, JPasswordField, JScrollPane, JLabel)
SQLite	Lightweight embedded relational database
JDBC	Java Database Connectivity for SQL operations via sqlite-jdbc driver
SHA-256	Cryptographic hashing for secure password storage (java.security.MessageDigest)
JUnit 5	Unit testing framework for automated testing of service layer methods
Git	Version control for source code management

3. Database Design

The application uses an SQLite database with 4 tables. The database file (trainbooking.db) is created automatically when the application first runs. Foreign keys are enabled via PRAGMA to make sure there is referential integrity.

3.1 ER Diagram

The entity-relationship diagram below shows the four tables and how they relate to each other. The bookings table serves as the central junction, linking users, trains, and seats together through foreign key relationships.



Relationships:

- users (1) -> (many) bookings
- trains (1) -> (many) bookings
- trains (1) -> (many) seats
- seats (1) -> (many) bookings

3.2 Database Tables (Structure & Data)

users

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique user identifier
username	TEXT	NOT NULL, UNIQUE	Login username
password_hash	TEXT	NOT NULL	SHA-256 hash of password
full_name	TEXT	NOT NULL	Display name
email	TEXT	NOT NULL, UNIQUE	User email address

	id ▲	username	password_hash	full_name	email
	Filter	Filter	Filter	Filter	Filter
1	1	javieramiri	8d969eef6ecad3c29a3a629280e686cf0c3f…	javier	javier@email.com
2	2	aaronT	f64561e04c3be9cea6271afcd2b324f4b865…	Aaron Tireny	aaron@yahoo.com
3	3	Maduro	af32a518fa35badd45cc983585b341ea6cab…	Nicolas Maduro	presidente@gmail.com

Trains

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique train identifier
train_number	TEXT	NOT NULL, UNIQUE	e.g. IE101
train_name	TEXT	NOT NULL	e.g. Dublin Express
origin	TEXT	NOT NULL	Departure station
destination	TEXT	NOT NULL	Arrival station
departure_time	TEXT	NOT NULL	Departure datetime
arrival_time	TEXT	NOT NULL	Arrival datetime
total_seats	INTEGER	NOT NULL	Capacity of the train
price_per_seat	REAL	NOT NULL	Base price in EUR

	id	train_number	train_name	origin	destination	departure_time	arrival_time	total_seats	price_per_seat
	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	IE101	Dublin Express	Dublin	Cork	2025-05-01 07:00:00	2025-05-01 09:30:00	60	35.0
2	2	IE102	Cork Morning Star	Cork	Dublin	2025-05-01 08:00:00	2025-05-01 10:30:00	60	35.0
3	3	IE103	Galway Flyer	Dublin	Galway	2025-05-01 09:00:00	2025-05-01 11:15:00	80	28.0
4	4	IE104	Belfast Express	Dublin	Belfast	2025-05-01 10:00:00	2025-05-01 12:00:00	100	22.0
5	5	IE105	Limerick Connect	Dublin	Limerick	2025-05-01 11:00:00	2025-05-01 13:00:00	60	30.0
6	6	IE106	Waterford Intercity	Dublin	Waterford	2025-05-01 12:00:00	2025-05-01 14:00:00	60	25.0
7	7	IE107	Sligo Arrow	Dublin	Sligo	2025-05-01 13:00:00	2025-05-01 15:30:00	50	32.0
8	8	IE108	Evening Express	Dublin	Cork	2025-05-01 17:00:00	2025-05-01 19:30:00	60	35.0

Seats

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique seat identifier
train_id	INTEGER	NOT NULL, FK → trains(id)	Which train this seat belongs to
seat_number	TEXT	NOT NULL	e.g. 1A, 3C
seat_class	TEXT	NOT NULL, DEFAULT STANDARD	FIRST / SECOND / STANDARD
is_booked	INTEGER	NOT NULL, DEFAULT 0	0 = available, 1 = booked

	id	*train_id*	seat_number	seat_class	is_booked
	Filter	Filter	Filter	Filter	Filter
1	1	1	1A	FIRST	0
2	2	1	1B	FIRST	0
3	3	1	1C	FIRST	0
4	4	1	1D	FIRST	0
5	5	1	2A	FIRST	0
6	6	1	2B	FIRST	0
7	7	1	2C	FIRST	0

Bookings

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique booking identifier
user_id	INTEGER	NOT NULL, FK → users(id)	Who made the booking
train_id	INTEGER	NOT NULL, FK → trains(id)	Which train
seat_id	INTEGER	NOT NULL, FK → seats(id)	Which seat
status	TEXT	NOT NULL, DEFAULT CONFIRMED	CONFIRMED or CANCELLED
booked_at	TEXT	NOT NULL	Timestamp of booking
total_price	REAL	NOT NULL	Final price paid in EUR

	id	user_id	train_id	seat_id	status	booked_at	total_price
	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	1	2	97	CONFIRMED	2026-04-20 16:23:01	35.0
2	2	1	2	110	CONFIRMED	2026-04-20 16:23:41	35.0
3	3	2	4	254	CANCELLED	2026-04-21 17:55:38	22.0
4	4	2	4	201	CONFIRMED	2026-04-21 17:55:49	33.0
5	5	2	8	485	CANCELLED	2026-04-21 17:56:58	42.0

4. Application Architecture

I organised the code into four packages to keep things separate and easier to manage.

Package	Responsibility
com.trainbooking.model	Data model classes (User, Train, Seat, Booking) — plain Java objects representing database entities
com.trainbooking.db	Database access layer (DatabaseManager) — handles all SQL queries, table creation, and seed data using a singleton pattern
com.trainbooking.service	Business logic layer (UserService, TrainService, BookingService) — validation, hashing, pricing, and orchestration of database calls
com.trainbooking.gui	Presentation layer (MainFrame, LoginPanel, SearchPanel, SeatSelectionPanel, BookingsPanel) — all Swing GUI components

The MainFrame is the main window. It uses CardLayout to switch between the Login, Search, Seat Selection and Bookings screens. After logging in, a navigation bar appears at the top so you can move between screens and log out.

The GUI never talks to the database directly — everything goes through the service classes first, which handle validation before passing it on to the DatabaseManager.

The test folder contains the JUnit 5 tests in BookingServiceTest.java.

5. GUI Design

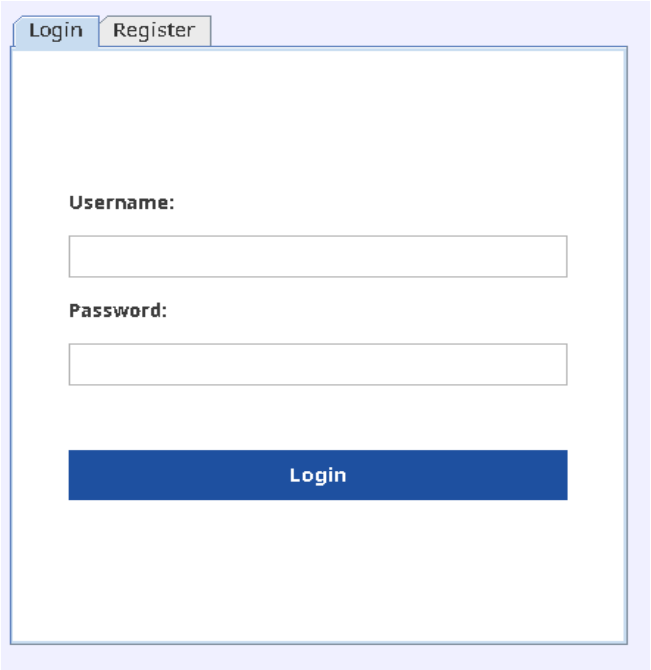
The app uses a range of Java Swing components for the interface. Each button uses an ActionListener to handle click events.

Swing Components Used

Component	Where / How It Is Used
JFrame	Main application window (MainFrame)
JPanel	Container for all screens, nav bar, form sections, legend row
JButton	Login, Register, Search, Select Seats, Confirm Booking, Cancel, Refresh, Logout, filter buttons, and seat grid buttons
JTextField	Username, full name, email, and date input fields
JPasswordField	Password and confirm password inputs (masks input with dots)
JComboBox	Origin and destination station dropdowns on search screen
JTable	Search results table and bookings table with custom cell renderers
JTabbedPane	Login/Register tab switcher on the login screen
JOptionPane	Confirmation dialogs (booking, cancellation, logout) and success/error messages
JScrollPane	Wraps the seat grid and both data tables for scrollable content
JLabel	Form labels, welcome message, page titles, legend items, status/error messages
CardLayout	Enables switching between Login, Search, Seats, and Bookings screens without new windows
GridBagLayout	Precise form layout on Login and Register panels
GridLayout	Seat grid layout (rows × columns of seat buttons)
BorderLayout	Main structure of each screen (NORTH/CENTER/SOUTH regions)
FlowLayout	Navigation bar buttons, search form row, legend items, filter buttons
ActionListener	All button click events use anonymous inner class implementation of ActionListener interface

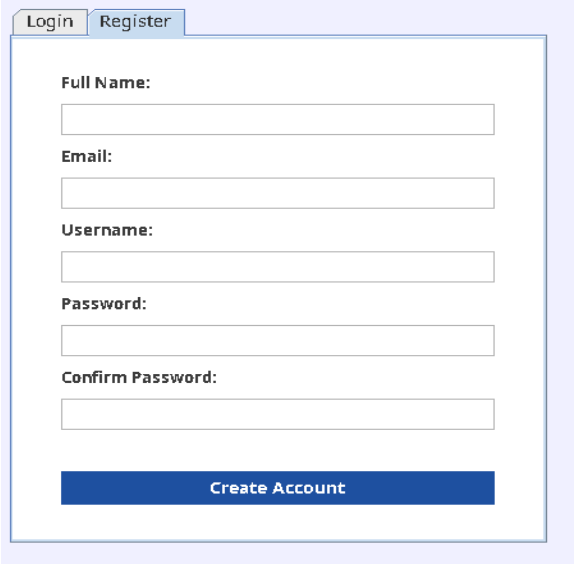
Application Screenshots

Login



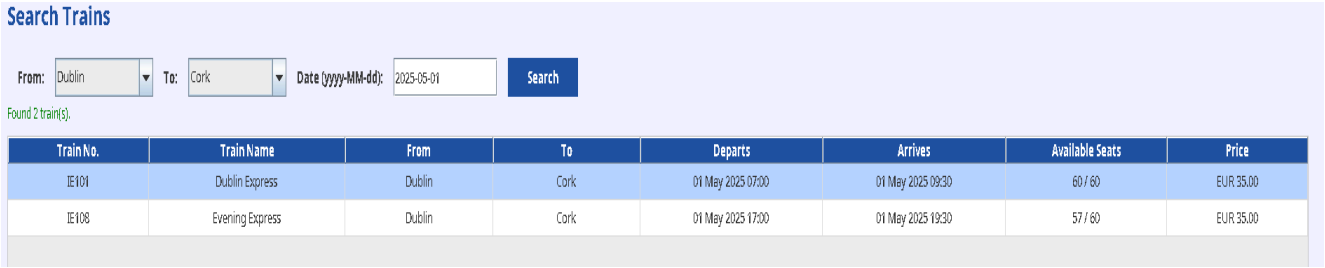
The login screen shows a tabbed interface with Login and Register tabs.

Register



The registration tab allows new users to create an account.

Search Trains

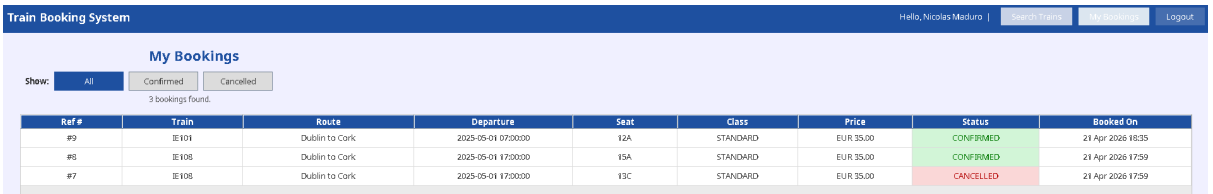


The search screen has dropdown menus for origin and destination, a date field, and displays matching trains in a table.

Seat Selection

The seat selection screen displays a colour-coded grid of seats. Green for Standard, gold for First class, blue for Second class, and red for already-booked seats.

My Bookings



The bookings screen shows all user bookings in a table with filter buttons. The status column is colour-coded green for confirmed and red for cancelled.

6. Interesting Source Code Snippets

This section demonstrates important design patterns, database techniques, and error handling used throughout the project.

6.1 Singleton Database Manager

The DatabaseManager uses the Singleton pattern to ensure only one database connection exists at a time across the entire application.

```
private static DatabaseManager instance;

private DatabaseManager() {}

public static DatabaseManager getInstance() {
   if (instance == null) {
      instance = new DatabaseManager();
   }
   return instance;
}
```

6.2 Password Hashing with SHA-256

Passwords are never stored in plain text. The UserService converts each password to a SHA-256 hash before storing it in the database.

```
private String hashPassword(String password) {
   try {
      MessageDigest digest = MessageDigest.getInstance("SHA-256");
      byte[] hashBytes = digest.digest(password.getBytes());
      StringBuilder hexString = new StringBuilder();
      for (int i = 0; i < hashBytes.length; i++) {
         String hex = Integer.toHexString(0xff & hashBytes[i]);
         if (hex.length() == 1) hexString.append('0');
         hexString.append(hex);
      }
      return hexString.toString();
   } catch (NoSuchAlgorithmException e) {
      return null;
   }
}
```

6.3 Seat Grid Event Handling

Each seat button has a click listener attached to it that handles seat selection. The final keyword is used on the local variables clickedSeat and clickedBtn so they can be accessed inside the listener, since local variables used inside an inner class must be final or effectively final in Java.

```
final Seat clickedSeat = seat;
final JButton clickedBtn = btn;
btn.addActionListener(new ActionListener() {
   public void actionPerformed(ActionEvent e) {
      handleSeatClick(clickedSeat, clickedBtn);
   }
});
```

6.4 Inner Join for Booking Details

When displaying a user's bookings, the system performs an inner join across the bookings, trains, and seats tables to retrieve all needed information in a single query.

```
String sql = "SELECT b.*, t.train_number, t.train_name, " +
   "t.origin, t.destination, " +
   "t.departure_time, s.seat_number, s.seat_class " +
   "FROM bookings b " +
   "JOIN trains t ON b.train_id = t.id " +
   "JOIN seats s ON b.seat_id = s.id " +
   "WHERE b.user_id = ? " +
   "ORDER BY b.booked_at DESC";
```

6.5 Input Validation and Error Handling

The UserService validates all registration inputs before attempting to write to the database. Each check returns a specific error message.

```
public String register(String username, String password,
                       String confirmPassword, String fullName,
                       String email) {
   if (username == null || username.trim().equals(""))
      return "Username is required.";
   if (username.trim().length() < 3)
      return "Username must be at least 3 characters.";
   if (password.length() < 6)
      return "Password must be at least 6 characters.";
   if (!password.equals(confirmPassword))
      return "Passwords do not match.";
   if (!email.contains("@"))
      return "Invalid email.";
   // ... hash and save to database
}
```

6.6 Finally Block for Resource Cleanup

Database queries use try/finally blocks to ensure PreparedStatement and ResultSet objects are always closed, even if an exception occurs. This follows the pattern taught in the JDBC lecture notes.

```
PreparedStatement ps = null;
ResultSet rs = null;
try {
   ps = connection.prepareStatement(sql);
   ps.setString(1, origin);
   ps.setString(2, destination);
   ps.setString(3, date + "%");
   rs = ps.executeQuery();
   while (rs.next()) {
      trains.add(mapTrain(rs));
   }
} finally {
   if (rs != null) rs.close();
   if (ps != null) ps.close();
}
```

7. Tests

7.1 JUnit Unit Tests

The project includes a JUnit 5 test class (BookingServiceTest.java) that automatically tests the service layer methods. The test class follows the structure taught in the Unit Testing lecture, using @BeforeAll, @BeforeEach, @AfterEach, @AfterAll annotations and assertEquals assertions.

The following unit tests are implemented:

Test Method	What It Tests
testCalculatePriceStandard()	Standard class returns base price unchanged
testCalculatePriceFirst()	First class multiplies base price by 1.5x
testCalculatePriceSecond()	Second class multiplies base price by 1.2x
testCalculatePriceRounding()	Price rounds correctly to 2 decimal places
testRegisterEmptyUsername()	Registration fails when username is empty
testRegisterShortPassword()	Registration fails when password is less than 6 characters
testRegisterPasswordMismatch()	Registration fails when passwords don't match
testRegisterInvalidEmail()	Registration fails when email has no @ symbol
testLoginEmptyUsername()	Login fails when username field is empty
testLoginEmptyPassword()	Login fails when password field is empty
testLoginWrongUsername()	Login fails when username does not exist
testSearchSameOriginDestination()	Search returns empty when origin equals destination
testSearchEmptyFields()	Search returns empty when fields are blank

Example test code from BookingServiceTest.java:

```
@Test
void testCalculatePriceFirst() {
   double result = bookingService.calculatePrice(35.00, "FIRST");
   assertEquals(52.50, result);
}

@Test
void testRegisterPasswordMismatch() {
   String result = userService.register("testuser",
      "password123", "password456", "Test User",
      "test@email.com");
   assertEquals("Passwords do not match.", result);
}
```

7.2 JUnit Test Results

Thanks for using JUnit! Support its development at https://junit.org/sponsoring

Setting up services for testing

Running: testCalculatePriceStandard()

Finished: testCalculatePriceStandard()

Running: testLoginEmptyUsername()

Finished: testLoginEmptyUsername()

Running: testSearchSameOriginDestination()

Finished: testSearchSameOriginDestination()

Running: testRegisterInvalidEmail()

Finished: testRegisterInvalidEmail()

Running: testCalculatePriceSecond()

Finished: testCalculatePriceSecond()

Running: testLoginWrongUsername()

Finished: testLoginWrongUsername()

Running: testLoginEmptyPassword()

Finished: testLoginEmptyPassword()

Running: testCalculatePriceFirst()

Finished: testCalculatePriceFirst()

Running: testRegisterShortPassword()

Finished: testRegisterShortPassword()

Running: testRegisterPasswordMismatch()

Finished: testRegisterPasswordMismatch()

Running: testCalculatePriceRounding()

Finished: testCalculatePriceRounding()

Running: testSearchEmptyFields()

Finished: testSearchEmptyFields()

Running: testRegisterEmptyUsername()

Finished: testRegisterEmptyUsername()

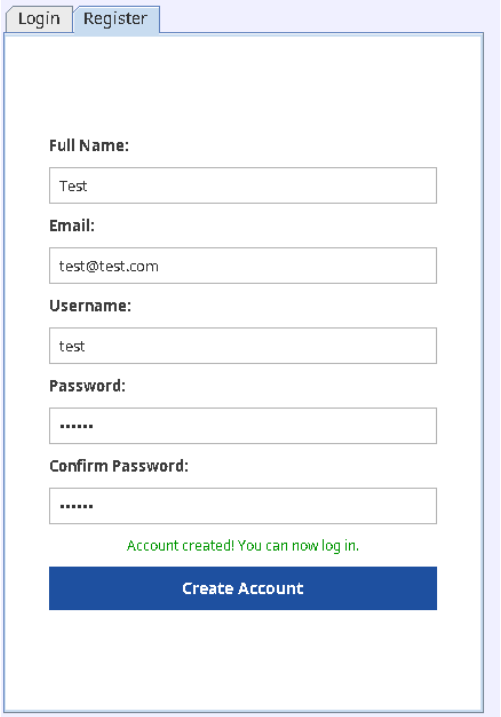
```
All tests complete
#[36m │ #[0m
#[36m├─#[0m #[36mJUnit Jupiter#[0m #[32m✔#[0m
#[36m│  └─#[0m #[36mBookingServiceTest#[0m #[32m✔#[0m
#[36m│    ├─#[0m #[34mtestCalculatePriceStandard()#[0m #[32m✔#[0m
#[36m│    ├─#[0m #[34mtestLoginEmptyUsername()#[0m #[32m✔#[0m
#[36m│    ├─#[0m #[34mtestSearchSameOriginDestination()#[0m #[32m✔#[0m
#[36m│    ├─#[0m #[34mtestRegisterInvalidEmail()#[0m #[32m✔#[0m
#[36m│    ├─#[0m #[34mtestCalculatePriceSecond()#[0m #[32m✔#[0m
#[36m│    ├─#[0m #[34mtestLoginWrongUsername()#[0m #[32m✔#[0m
#[36m│    ├─#[0m #[34mtestLoginEmptyPassword()#[0m #[32m✔#[0m
#[36m│    ├─#[0m #[34mtestCalculatePriceFirst()#[0m #[32m✔#[0m
#[36m│    ├─#[0m #[34mtestRegisterShortPassword()#[0m #[32m✔#[0m
#[36m│    ├─#[0m #[34mtestRegisterPasswordMismatch()#[0m #[32m✔#[0m
#[36m│    ├─#[0m #[34mtestCalculatePriceRounding()#[0m #[32m✔#[0m
#[36m│    ├─#[0m #[34mtestSearchEmptyFields()#[0m #[32m✔#[0m
#[36m│    └─#[0m #[34mtestRegisterEmptyUsername()#[0m #[32m✔#[0m
#[36m├─#[0m #[36mJUnit Vintage#[0m #[32m✔#[0m
#[36m└─#[0m #[36mJUnit Platform Suite#[0m #[32m✔#[0m

Test run finished after 94 ms
[         4 containers found      ]
[         0 containers skipped    ]
[         4 containers started    ]
[         0 containers aborted    ]
[         4 containers successful ]
[         0 containers failed     ]
[        13 tests found           ]
[         0 tests skipped         ]
[        13 tests started         ]
[         0 tests aborted         ]
[        13 tests successful      ]
[         0 tests failed          ]
```

7.3 Manual Test Cases

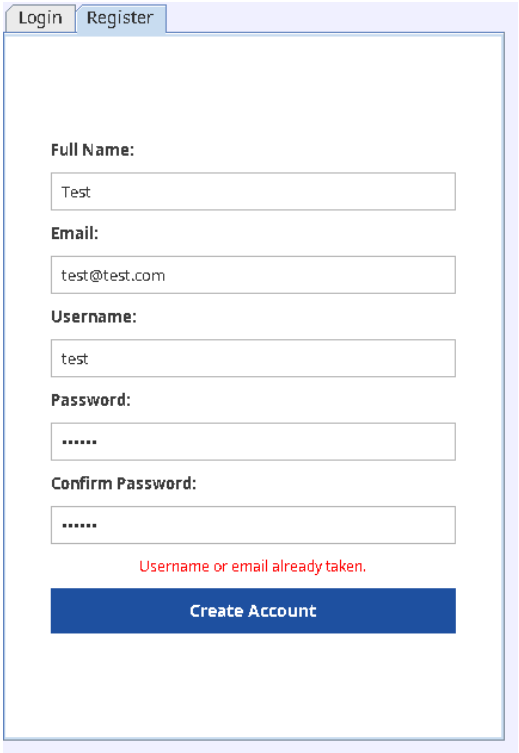
In addition to JUnit tests, the application was tested manually to verify that all CRUD operations and GUI interactions work correctly:

#	Test Case	Steps	Expected Result
1	Register with valid details	Enter valid username, password, name, email and click Create Account	Success message, can log in
2	Register duplicate username	Try to register with an existing username	Error: username or email taken
3	Register with bad email	Enter email without @ symbol	Error: Invalid email
4	Login with correct credentials	Enter valid username and password	Redirected to search screen
5	Login with wrong password	Enter valid username, wrong password	Error: Wrong password
6	Search for trains	Select origin, destination, date, click Search	Matching trains shown in table
7	Search same origin/destination	Select same station for From and To	Error: can't be the same
8	Book a seat	Select train, choose seat, confirm	Booking confirmed, seat turns red
9	Book without selecting seat	Click Confirm without picking a seat	Error: Pick a seat first
10	View bookings	Navigate to My Bookings	All bookings shown correctly
11	Filter bookings	Click Confirmed / Cancelled filters	Table filters correctly
12	Cancel a booking	Select confirmed booking, click Cancel	Status changes to CANCELLED
13	Cancel already cancelled	Select cancelled booking, click Cancel	Error: Already cancelled
14	Logout	Click Logout, confirm	Returned to login screen

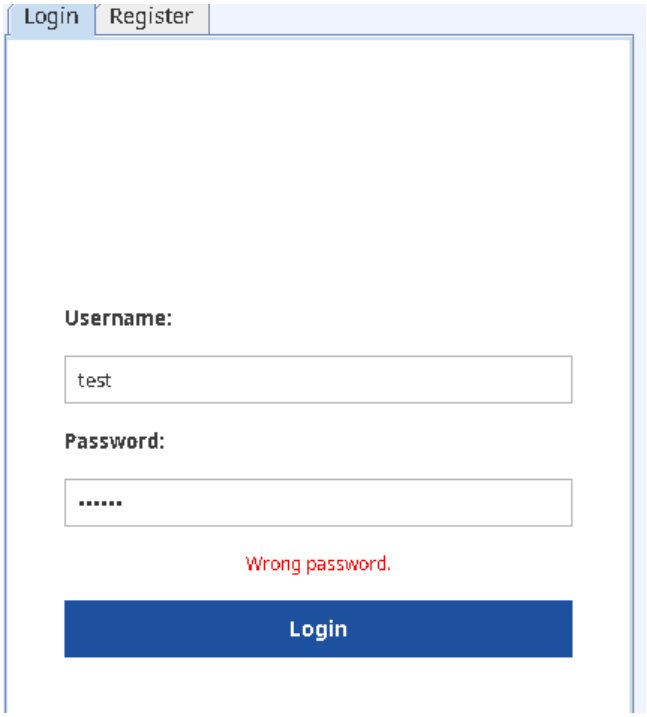
Register with valid details



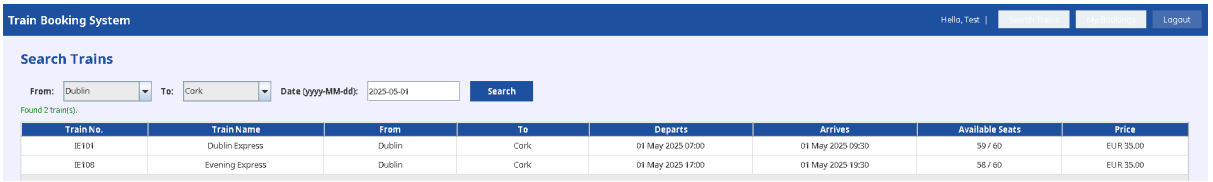
Register with duplicate username



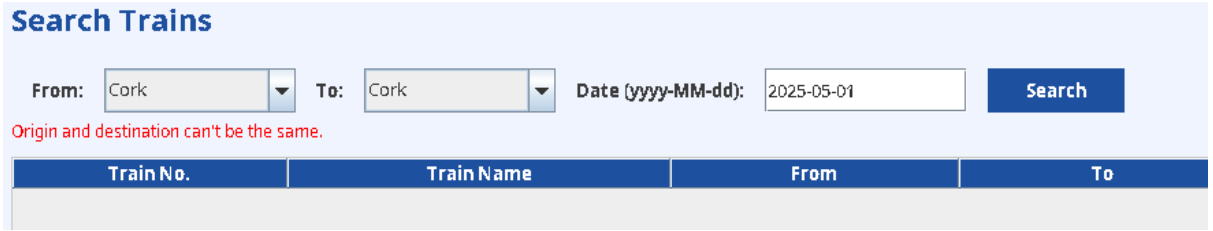
Login with wrong password



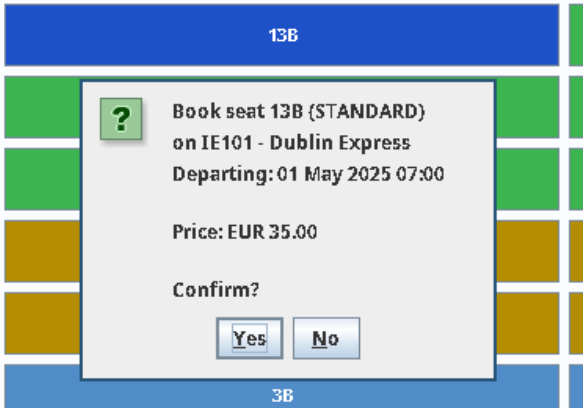
Train Search



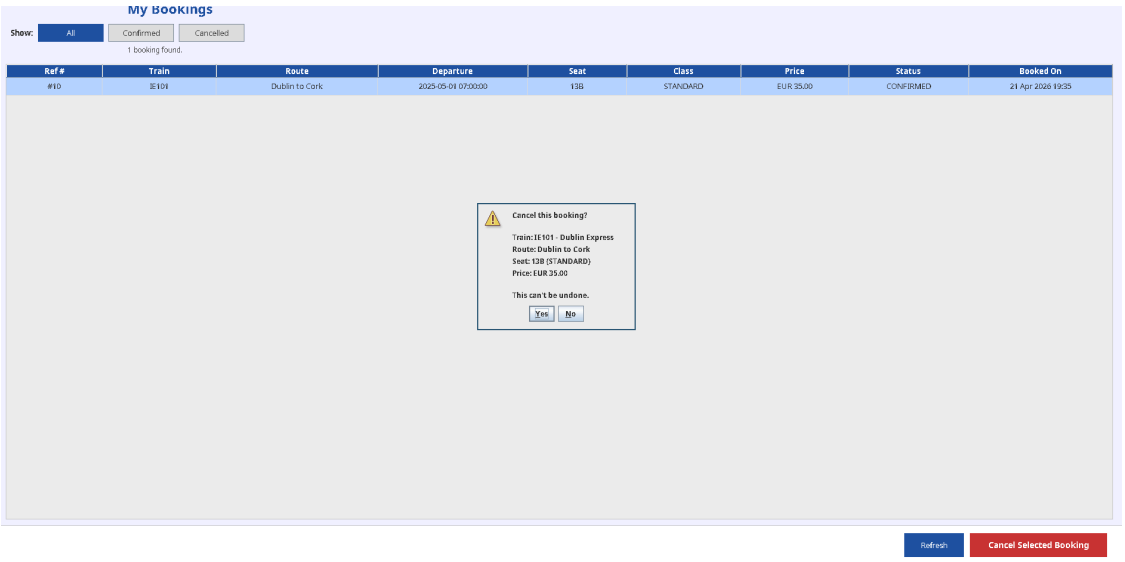
Train Search with the same origin and destination

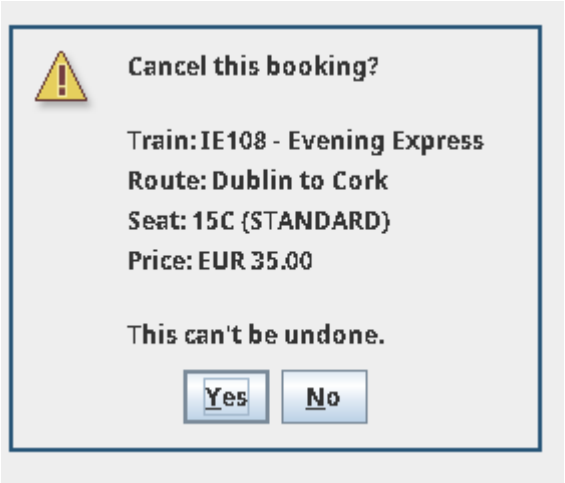


Booking Seat Message



Cancel Booking





9. References

Barron, J. (2023) JDBC Lecture Notes. South East Technological University, Department of Computing.

Barron, J. (2023) Implementing GUIs Lecture Notes. South East Technological University, Department of Computing.

Barron, J. (2024) Unit Testing Lecture Notes. South East Technological University, Department of Computing.

Oracle (2024) Java SE 17 Documentation. Available at: https://docs.oracle.com/en/java/ [Accessed: April 2026].

Oracle (2024) Class MessageDigest. Available at: https://docs.oracle.com/javase/8/docs/api/java/security/MessageDigest.html [Accessed: April 2026].

JUnit Team (2024) JUnit 5 User Guide. Available at: https://junit.org/junit5/docs/current/user-guide/ [Accessed: April 2026].

Xerial (2024) sqlite-jdbc. Available at: https://github.com/xerial/sqlite-jdbc [Accessed: April 2026].

SQLite (2024) SQLite Documentation. Available at: https://www.sqlite.org/docs.html [Accessed: April 2026].